

Real-world (make it feel alive with streaming) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Work through in order — don't jump ahead until you've genuinely tried.

- [Hint 1](#)
- [Hint 2](#)
- [Hint 3](#)
- [Hint 4](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 1

Simple Version

Add one argument to the same call you already know: `stream=True`. Instead of getting one finished answer back, you get back something you loop over — a series of small pieces of the answer, one at a time, in order.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

With `stream=True`, `client.chat.completions.create(...)` returns an iterable of "chunk" objects instead of one finished response object. Each chunk carries a small piece of the answer at `chunk.choices[0].delta.content` — and this can be `None` (for chunks that don't carry new text, like the very first one), so you need to guard against printing `None`.

Sketch the loop structure (in plain English) before moving to Hint 2: what do you do with each chunk as it arrives?

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 2

Simple Version

Use `print(piece, end="")` instead of a normal `print(piece)` — the default `print()` adds a newline after every call, which would put each little piece of text on its own line instead of joining them into one flowing sentence.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

`print(text, end="")` — the `end=""` argument replaces the default trailing newline with nothing, so consecutive `print()` calls flow together on the same line, which is what makes streamed output look like it's "typing." Also handle the `None` case cleanly: `piece = chunk.choices[0].delta.content` can be `None`, so check `if piece:` before printing, or use `piece or ""` so you're never trying to print `None` itself.

If you also want to build up the full reply text (useful if you need to save it into conversation history afterward, like in the previous exercise), start with an empty string before the loop and add each piece to it as it arrives.

Write the full loop now before checking Hint 3.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 3

Simple Version

```
call the model with stream=True
for each piece that comes back:
    get the text out of this piece
    if there is text: print it, with no newline after
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
stream = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=history,
    stream=True,
)

full_reply = ""
for chunk in stream:
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece, end="")
        full_reply = full_reply + piece

print() # move to a new line once the reply is finished
```

`full_reply` accumulates the whole answer as it streams by, so once the loop finishes you have the complete text — useful if you then need to append it to `history` the way the previous exercise did. The final bare `print()` just adds a newline after the streamed text finishes, so the next thing you print doesn't run onto the same line.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 4

Simple Version

Try it on a long-ish question and actually watch the difference — run the same question once streamed and once not. Notice how much faster the *first word* appears with streaming, even though the *whole* answer finishes around the same total time either way.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

Run a direct side-by-side comparison: time (roughly, by eye) how long it takes for text to first appear on screen in the streaming version versus the non-streaming version from the earlier exercises. The gap in *perceived* speed comes entirely from when the first token shows up, not from the total generation time — the model isn't actually answering any faster, it's just showing you its work in progress instead of making you wait for all of it at once.

Try this comparison now, then check your streaming loop against the full [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Full solution: [Show me the solution](#)